0.57. Un carré sur la pointe : losange

losange(x, y, tuile, taille, sens, vitesse, tours) : le carré tourné, ses côtés en diagonale. Les mêmes réglages que carre.

1. Ce qu'il faut comprendre

Un losange, c'est un carré posé sur la pointe. Ses quatre côtés ne sont plus droits : ce sont des **diagonales**. À chaque pas, la lettre bouge d'une case sur X **et** d'une case sur Y en même temps, comme `deplace(x, y, …, 5, 5)` au 0.26.

Ce qui est nouveau ici : `losange(x, y, tuile, taille, sens, vitesse, tours)`. Les **mêmes sept réglages que** `carre`, dans le même ordre.

• `x`, `y` : le centre. `taille` : la distance du centre à chaque **pointe**. Avec 2, les pointes sont à 2 cases en haut, à droite, en bas et à gauche du centre.

• `sens` : `1` part vers la droite depuis la pointe du haut (comme les aiguilles d'une montre) ; `-1` part vers la gauche.

• `vitesse` en millisecondes par pas, `tours` : comme pour `carre`.

Le trajet : du centre, la lettre monte **tout droit** jusqu'à la pointe du haut ; elle fait le tour en diagonale ; puis elle redescend tout droit au centre.

Un Carre marche aussi : puisque les réglages sont les mêmes, `losange(ronde);` accepte un `Carre ronde = { … };`, comme `carre(ronde);`.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <losange>` → 0.56.3. La fonction losange() — tourner en losange
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <losange>  // une tuile qui tourne en losange`	Demande à la console « losange » (ce que c'est : une tuile qui tourne en losange). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en losange ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// Un losange autour de la case (10, 8). Mêmes réglages que carre :`	Un commentaire : une phrase pour celui qui lit (« Un losange autour de la case (10, 8). Mêmes réglages que carre : »). La console l'ignore.
`//  losange(10, 8, ALPHABET[0], 2, 1, 250, 1);`	Un commentaire : une phrase pour celui qui lit (« losange(10, 8, ALPHABET[0], 2, 1, 250, 1); »). La console l'ignore.

`//  |  |  |  |  |  |  |`	Un commentaire : une phrase pour celui qui lit (« ||||||| »). La console l'ignore.
`//  |  |  |  |  |  |  +-- tours : 1 tour`	Un commentaire : une phrase pour celui qui lit (« ||||| +-- tours : 1 tour »). La console l'ignore.
`//  |  |  |  |  |  +------- vitesse : 250 ms par pas`	Un commentaire : une phrase pour celui qui lit (« |||| +------- vitesse : 250 ms par pas »). La console l'ignore.
`//  |  |  |  |  +---------- sens : 1, vers la droite d'abord`	Un commentaire : une phrase pour celui qui lit (« ||| +---------- sens : 1, vers la droite d'abord »). La console l'ignore.
`//  |  |  |  +------------- taille : 2, les pointes à 2 cases`	Un commentaire : une phrase pour celui qui lit (« || +------------- taille : 2, les pointes à 2 cases »). La console l'ignore.
`//  |  |  +-------------------------- tuile : la lettre A`	Un commentaire : une phrase pour celui qui lit (« | +---------- tuile : la lettre A »). La console l'ignore.
`//  +---+---------------------------- x, y : le centre (10, 8)`	Un commentaire : une phrase pour celui qui lit (« +---+---------- x, y : le centre (10, 8) »). La console l'ignore.
`// Le chemin (sens 1). Chaque flèche est UN pas en diagonale :`	Un commentaire : une phrase pour celui qui lit (« Le chemin (sens 1). Chaque flèche est UN pas en diagonale : »). La console l'ignore.
`//  colonne :  8  9  10  11  12`	Un commentaire : une phrase pour celui qui lit (« colonne : 8 9 10 11 12 »). La console l'ignore.
`//  ligne 6 :  2  2 : la pointe du haut`	Un commentaire : une phrase pour celui qui lit (« ligne 6 : 2 2 : la pointe du haut »). La console l'ignore.
`//  ligne 7 :  ↗  ↘`	Un commentaire : une phrase pour celui qui lit (« ligne 7 : ↗ ↘ »). La console l'ignore.
`//  ligne 8 :  ↑  A  4  A : le centre, où tout commence`	Un commentaire : une phrase pour celui qui lit (« ligne 8 : ↑ A 4 A : le centre, où tout commence »). La console l'ignore.
`//  ligne 9 :  ↖  ↙ et où tout finit`	Un commentaire : une phrase pour celui qui lit (« ligne 9 : ↖ ↙ et où tout finit »). La console l'ignore.
`//  ligne 10 :  6`	Un commentaire : une phrase pour celui qui lit (« ligne 10 : 6 »). La console l'ignore.
`//  1. du centre (10, 8), tout droit vers le haut, jusqu'à la pointe (10, 6)`	Un commentaire : une phrase pour celui qui lit (« 1. du centre (10, 8), tout droit vers le haut, jusqu'à la pointe (10, 6) »). La console l'ignore.
`//  2. en diagonale, bas-droite, jusqu'à la pointe de droite (12, 8)`	Un commentaire : une phrase pour celui qui lit (« 2. en diagonale, bas-droite, jusqu'à la pointe de droite (12, 8) »). La console l'ignore.
`//  3. bas-gauche, jusqu'à la pointe du bas (10, 10)`	Un commentaire : une phrase pour celui qui lit (« 3. bas-gauche, jusqu'à la pointe du bas (10, 10) »). La console l'ignore.
`//  4. haut-gauche, jusqu'à la pointe de gauche (8, 8)`	Un commentaire : une phrase pour celui qui lit (« 4. haut-gauche, jusqu'à la pointe de gauche (8, 8) »). La console l'ignore.
`//  5. haut-droite, retour à la pointe du haut (10, 6)`	Un commentaire : une phrase pour celui qui lit (« 5. haut-droite, retour à la pointe du haut (10, 6) »). La console l'ignore.
`//  6. tout droit vers le bas, retour au centre (10, 8)`	Un commentaire : une phrase pour celui qui lit (« 6. tout droit vers le bas, retour au centre (10, 8) »). La console l'ignore.
`losange(10, 8, ALPHABET[0], 2, 1, 250, 1);`	Appel de la fonction losange de la console, avec les arguments 10, 8, ALPHABET[0], 2, 1, 250, 1. Ce qu'elle fait : le carré posé sur la

	pointe, ses côtés en diagonale, autour de (x, y) : les pointes sont à « taille » cases du centre. Mêmes réglages que carre() ; accepte aussi un Carre : losange(ronde);.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // le tour est fini : le A reste au centre`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « le tour est fini : le A reste au centre ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1358 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  losange()       449 o  ← main()  — une fonction fournie par le langage
  main()           38 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans losange()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans losange()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans losange(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  losange() × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <losange>    // une tuile qui tourne en losange
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

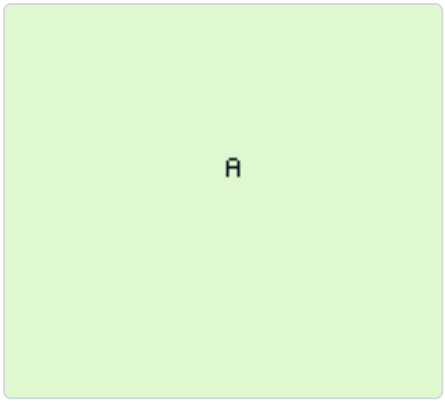
```
int main() {
  // Un losange autour de la case (10, 8). Mêmes réglages que carre :
  //
  //   losange(10, 8, ALPHABET[0], 2, 1, 250, 1);
  //           |   |  |            |  |  |    |
  //           |   |  |            |  |  |    +-- tours   : 1 tour
  //           |   |  |            |  |  +------- vitesse : 250 ms par pas
  //           |   |  |            |  +---------- sens    : 1, vers la droite d'abord
  //           |   |  |           +------------- taille  : 2, les pointes à 2 cases
  //           |   |  +-------------------------- tuile   : la lettre A
  //           +---+----------------------------- x, y    : le centre (10, 8)
  //
  // Le chemin (sens 1). Chaque flèche est UN pas en diagonale :
  //
  //      colonne :   8   9  10  11  12
  //      ligne 6 :           2               2 : la pointe du haut
  //      ligne 7 :       ↗       ↘
  //      ligne 8 :   ↑       A       4       A : le centre, où tout commence
  //      ligne 9 :       ↖       ↙               et où tout finit
  //      ligne 10 :          6
  //
  //   1. du centre (10, 8), tout droit vers le haut, jusqu'à la pointe (10, 6)
  //   2. en diagonale, bas-droite, jusqu'à la pointe de droite (12, 8)
  //   3. bas-gauche, jusqu'à la pointe du bas (10, 10)
  //   4. haut-gauche, jusqu'à la pointe de gauche (8, 8)
  //   5. haut-droite, retour à la pointe du haut (10, 6)
  //   6. tout droit vers le bas, retour au centre (10, 8)
  losange(10, 8, ALPHABET[0], 2, 1, 250, 1);

  while (true) {
    image();          // le tour est fini : le A reste au centre
  }
}
```

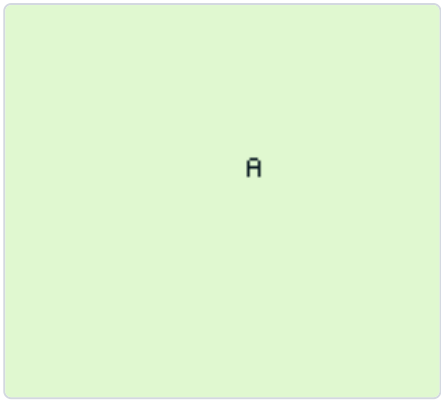
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

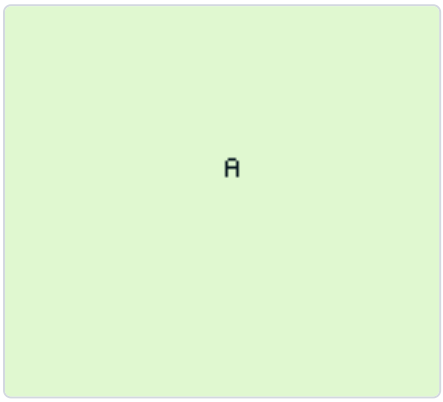
la toute première image — la console vient de démarrer — image 1



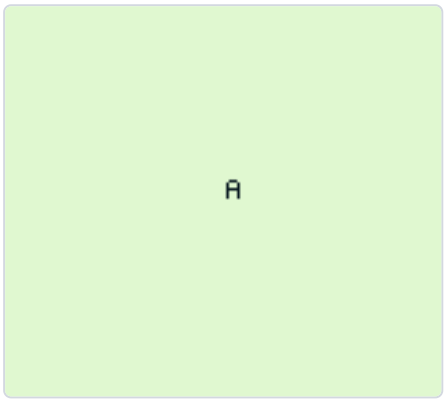
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Un A qui monte de 2 cases, fait un tour en diagonale autour de (10, 8), puis revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre pointes : (10, 6), (12, 8), (10, 10), (8, 8)
- ✓ les côtés en diagonale : (11, 7), (11, 9), (9, 9), (9, 7)
- ✓ sens 1 : après la pointe du haut, vers la droite
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.57.1. Un carré sur la pointe : losange — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++